1. The Basics of Database Structures

Relational Database Theory

- **Definition**: A relational database organizes data into tables, where each table (or relation) contains rows and columns. Tables are linked through specific columns called **keys** (primary and foreign keys) to establish relationships.
- **Key Concepts**:
  - **Tables**: Store data in rows (records) and columns (attributes).
  - **Primary Key**: A unique identifier for each row in a table (e.g., CustomerID).
  - **Foreign Key**: A column in one table that references the primary key of another table to link data.
  - **Relationships**: One-to-one, one-to-many, or many-to-many relationships between tables.
- **Why It Matters**: Relational databases ensure data integrity, reduce redundancy, and allow efficient querying using SQL.

The SQLite Database Engine

- **Definition**: SQLite is a lightweight, serverless, file-based relational database engine that supports SQL for managing data.
- **Key Features**:
  - **Self-Contained**: No separate server is required; the database is stored in a single file.
  - **Portable**: Ideal for small applications, mobile apps, or embedded systems.
  - **SQL Support**: Supports most SQL standards for querying and managing data.
- **Use Case**: Used in applications where simplicity and minimal setup are needed, such as in DB Browser for SQLite.

---

2. The SQL Stack

WSDA Music (Sample Database)

- **Definition**: WSDA Music is a sample database used for learning SQL, containing tables like customers, tracks, invoices, etc., to practice queries.
- **Purpose**: Provides a practical dataset to explore SQL operations like selecting, filtering, and joining tables.
- **Structure**: Includes related tables with data about music tracks, customers, and sales.

How to Install DB Browser for SQLite on Your Computer

- **Definition**: DB Browser for SQLite is a free, open-source tool for interacting with SQLite databases.
- **Steps**:
  a. Download the installer from the official DB Browser website.
  b. Run the installer and follow prompts (select OS: Windows, macOS, or Linux).
  c. Launch DB Browser to create or open SQLite database files.
- **Why Use It**: Provides a user-friendly interface to write SQL queries, view data, and manage databases.

3. The SQLite Database Environment

How to Access the DB Browser for SQLite Software

- **Steps**:
  a. Open DB Browser after installation.
  b. Create a new database or open an existing `.sqlite` file.
  c. Navigate the interface: tabs for browsing data, writing queries, and managing schema.
- **Purpose**: Familiarizes users with the DB Browser interface for SQL operations.

Load the Sample Database File

- **Process**:
  a. In DB Browser, select **File > Open Database**.
  b. Choose the WSDA Music `.sqlite` file.
  c. Verify that tables (e.g., tracks, customers) load correctly.
- **Outcome**: Allows users to work with a pre-built dataset for practice.

Getting Familiar with a Database

- **Steps**:
  – Explore the database schema (tables, columns, and relationships).
  – Check column data types (e.g., INTEGER, TEXT, DATE).
  – View sample data in tables to understand content.
- **Why It Matters**: Understanding the database structure is crucial before writing queries.

The Browse Data Area

- **Definition**: The "Browse Data" tab in DB Browser displays table contents in a spreadsheet-like view.
- **Features**:
  – View rows and columns of any table.
  – Sort or filter data directly in the interface.

- **Use Case**: Helps users visually inspect data before querying.

The Query Writing Area

- **Definition**: The "Execute SQL" tab in DB Browser where users write and run SQL queries.
- **Features**:
  - Write and execute SELECT, INSERT, UPDATE, or DELETE queries.
  - View query results below the input area.
  - Save queries for reuse.
- **Tip**: Test queries incrementally to avoid errors.

---

4. Composing Queries

Queries

- **Definition**: A query is an SQL statement used to retrieve, manipulate, or manage data in a database.
- **Basic Syntax**: `SELECT column_name FROM table_name;`
- **Example**: `SELECT FirstName, LastName FROM Customers;`

Query Commenting

- **Definition**: Comments in SQL explain query logic for clarity or documentation.
- **Syntax**:
  - Single-line: `-- Comment text`
  - Multi-line: `/* Comment text */`
- **Example**:
- ```
  -- Select customer names
  SELECT FirstName, LastName FROM Customers;
  ```
- **Why Use**: Improves readability and collaboration.

Query Composition

- **Definition**: Writing SQL queries to retrieve specific data by selecting columns, filtering rows, and sorting results.
- **Steps**:
  a. Identify the table(s) and column(s).
  b. Use `SELECT` to specify columns.
  c. Add conditions with `WHERE`, sorting with `ORDER BY`, etc.
- **Example**:
  ```

- `SELECT TrackName, UnitPrice FROM Tracks WHERE UnitPrice > 0.99;`

Query Composition Best Practices

- **Tips**:
  - Use clear column and table names.
  - Avoid selecting unnecessary columns (`SELECT *`).
  - Test queries on small datasets first.
  - Format queries for readability (e.g., align clauses).
- **Why It Matters**: Improves query efficiency and maintainability.

Column Custom Names

- **Definition**: Assigning aliases to columns in query results using `AS`.
- **Syntax**: `SELECT column_name AS alias_name FROM table_name;`
- **Example**:
- `SELECT FirstName AS Name, LastName AS Surname FROM Customers;`
- **Use Case**: Makes output more readable or application-friendly.

Sorting Query Results

- **Definition**: Arranging query results in a specific order using `ORDER BY`.
- **Syntax**: `SELECT column_name FROM table_name ORDER BY column_name [ASC|DESC];`
- **Example**:
- `SELECT TrackName, UnitPrice FROM Tracks ORDER BY UnitPrice DESC;`
- **Default**: Ascending (`ASC`); use `DESC` for descending.

Limiting Query Results

- **Definition**: Restricting the number of rows returned using `LIMIT`.
- **Syntax**: `SELECT column_name FROM table_name LIMIT n;`
- **Example**:
- `SELECT TrackName FROM Tracks LIMIT 10;`
- **Use Case**: Useful for testing or displaying top results.

Code Challenge: Concise Track Pricing Report

- **Task**: Write a query to display track names and prices, possibly sorted or limited.
- **Example Solution**:
- `SELECT TrackName, UnitPrice FROM Tracks ORDER BY UnitPrice DESC LIMIT 5;`

---

5. Discovering Insights in Data

Types of SQL Operators

- **Definition**: Operators perform comparisons or calculations in SQL queries.
- **Types**:
  - **Arithmetic**: `+`, `-`, `*`, `/`, `%`
  - **Comparison**: `=`, `!=`, `<`, `>`, `<=`, `>=`
  - **Logical**: `AND`, `OR`, `NOT`
- **Example**: `SELECT TrackName FROM Tracks WHERE UnitPrice > 0.99 AND GenreId = 1;`

Filter and Analyze Numeric Data

- **Definition**: Use comparison operators to filter numeric columns.
- **Example**:
- `SELECT TrackName, UnitPrice FROM Tracks WHERE UnitPrice >= 1.00;`
- **Use Case**: Identify tracks above a certain price.

BETWEEN and IN Operators

- **BETWEEN**: Filters values within a range.
  - Syntax: `WHERE column_name BETWEEN value1 AND value2;`
  - Example: `SELECT TrackName FROM Tracks WHERE UnitPrice BETWEEN 0.99 AND 1.99;`
- **IN**: Filters values matching a list.
  - Syntax: `WHERE column_name IN (value1, value2, ...);`
  - Example: `SELECT TrackName FROM Tracks WHERE GenreId IN (1, 2, 3);`

Filter and Analyze Text Data

- **Definition**: Use operators like `=` or `LIKE` to filter text columns.
- **Example**:

- `SELECT FirstName FROM Customers WHERE FirstName = 'John';`

Search Records Without an Exact Match

- **Definition**: Use `LIKE` with wildcards (`%` for any characters, `_` for one character).
- **Example**:
- `SELECT TrackName FROM Tracks WHERE TrackName LIKE 'Rock%';`
- **Use Case**: Find tracks starting with "Rock".

Filter and Analyze Using Dates

- **Definition**: Filter records based on date columns using comparison operators or date functions.
- **Example**:
- `SELECT InvoiceDate FROM Invoices WHERE InvoiceDate >= '2023-01-01';`

Filter Records Based on More Than One Condition

- **Definition**: Combine conditions using `AND`, `OR`, or `NOT`.
- **Example**:
- `SELECT TrackName FROM Tracks WHERE UnitPrice > 0.99 AND GenreId = 1;`

Logical Operator OR

- **Definition**: Retrieves rows where at least one condition is true.
- **Example**:
- `SELECT TrackName FROM Tracks WHERE GenreId = 1 OR GenreId = 2;`

Brackets and Order

- **Definition**: Use parentheses to control the order of logical operations.
- **Example**:
- `SELECT TrackName FROM Tracks WHERE (UnitPrice > 0.99 OR GenreId = 1) AND AlbumId = 5;`
- **Why Use**: Ensures correct evaluation of complex conditions.

IF THEN Logic with CASE

- **Definition**: The `CASE` statement adds conditional logic to queries.
- **Syntax**:
- ```
  SELECT column_name,
  CASE
  WHEN condition THEN result1
  ELSE result2
  END AS alias_name
  FROM table_name;
  ```
- **Example**:
- ```
  SELECT TrackName,
  CASE
  WHEN UnitPrice > 1.00 THEN 'Premium'
  ELSE 'Standard'
  END AS PriceCategory
  FROM Tracks;
  ```

Code Challenge: Categorize Tracks by Price

- **Task**: Use `CASE` to categorize tracks based on price thresholds.
- **Example Solution**:
- ```
  SELECT TrackName, UnitPrice,
  CASE
  WHEN UnitPrice > 1.50 THEN 'High'
  WHEN UnitPrice > 1.00 THEN 'Medium'
  ELSE 'Low'
  END AS PriceTier
  FROM Tracks;
  ```

---

6. Accessing Data from Multiple Tables

Joins Explained

- **Definition**: Joins combine data from multiple tables based on related columns.
- **Purpose**: Retrieve related data across tables (e.g., customer names and their orders).

How Tables Share a Relationship, Part 1 & 2

- **Definition**: Relationships are defined by primary and foreign keys.
- **Example**:
  ```

  - `Customers` table: `CustomerId` (primary key).
  - `Invoices` table: `CustomerId` (foreign key) links to `Customers`.
- **Types**:
  - **One-to-Many**: One customer has many invoices.
  - **Many-to-Many**: Tracks and playlists (via a junction table).

Simplifying JOINs

- **Definition**: Use clear syntax and aliases to make joins readable.
- **Example**:
- ```
  SELECT c.FirstName, i.InvoiceId
  FROM Customers c
  JOIN Invoices i ON c.CustomerId = i.CustomerId;
  ```

Types of JOINs

- **INNER JOIN**: Returns only matching rows from both tables.
- **LEFT JOIN**: Returns all rows from the left table, with NULLs for non-matching rows in the right table.
- **RIGHT JOIN**: Returns all rows from the right table, with NULLs for non-matching rows in the left table.
- **FULL JOIN**: Returns all rows from both tables, with NULLs for non-matches (not supported in SQLite).

The INNER JOIN

- **Syntax**:
- `SELECT columns FROM table1 INNER JOIN table2 ON table1.column = table2.column;`
- **Example**:
- ```
  SELECT c.FirstName, i.InvoiceDate
  FROM Customers c
  INNER JOIN Invoices i ON c.CustomerId = i.CustomerId;
  ```

The LEFT JOIN

- **Syntax**:
- `SELECT columns FROM table1 LEFT JOIN table2 ON table1.column = table2.column;`
- **Example**:
- ```
  SELECT c.FirstName, i.InvoiceId
  FROM Customers c
  LEFT JOIN Invoices i ON c.CustomerId = i.CustomerId;
  ```
  ```

The RIGHT JOIN

- **Syntax**:
- `SELECT columns FROM table1 RIGHT JOIN table2 ON table1.column = table2.column;`
- **Note**: SQLite does not support `RIGHT JOIN` natively; rewrite as `LEFT JOIN` by swapping tables.

Tables and Entity Relationship Diagrams

- **Definition**: ERDs visually represent tables, columns, and relationships (primary/foreign keys).
- **Use Case**: Helps plan and understand database structure before querying.

Joining Many Tables

- **Definition**: Combine multiple tables using multiple `JOIN` clauses.
- **Example**:
- ```
  SELECT c.FirstName, i.InvoiceId, t.TrackName
  FROM Customers c
  JOIN Invoices i ON c.CustomerId = i.CustomerId
  JOIN Invoice_Items ii ON i.InvoiceId = ii.InvoiceId
  JOIN Tracks t ON ii.TrackId = t.TrackId;
  ```

Code Challenge: Analyzing Customer Support Interactions

- **Task**: Join tables to analyze customer support data (e.g., customer names and support interactions).
- **Example Solution**:
- ```
  SELECT c.FirstName, c.LastName, s.SupportRepId
  FROM Customers c
  JOIN Employees s ON c.SupportRepId = s.EmployeeId;
  ```

---

7. SQL Functions

Calculating with Functions

- **Definition**: SQL functions perform operations on data (e.g., calculations, formatting).
- **Types**: Aggregate (e.g., `SUM`), scalar (e.g., `UPPER`), date functions.

String, Date, and Aggregate Function Types

- **String**: Manipulate text (e.g., `UPPER`, `CONCAT`).
- **Date**: Handle dates (e.g., `DATE`, `STRFTIME` in SQLite).
- **Aggregate**: Summarize data (e.g., `COUNT`, `AVG`).

Connecting Strings

- **Definition**: Combine strings using `||` (SQLite) or `CONCAT`.
- **Example**:
- `SELECT FirstName || ' ' || LastName AS FullName FROM Customers;`

Separating Text

- **Definition**: Extract parts of strings using `SUBSTR`.
- **Example**:
- `SELECT SUBSTR(FirstName, 1, 3) AS ShortName FROM Customers;`

UPPER and LOWER String Functions

- **Definition**: Convert text to uppercase (`UPPER`) or lowercase (`LOWER`).
- **Example**:
- `SELECT UPPER(FirstName) AS Name FROM Customers;`

Date Functions

- **Definition**: Manipulate dates (e.g., `STRFTIME` in SQLite for formatting).
- **Example**:
- `SELECT STRFTIME('%Y', InvoiceDate) AS Year FROM Invoices;`

Aggregate Functions

- **Definition**: Summarize data across rows.
- **Examples**:
  - `COUNT(*)`: Counts rows.
  - `SUM(UnitPrice)`: Totals a numeric column.
  - `AVG(UnitPrice)`: Averages a numeric column.
- **Example**:
- `SELECT COUNT(*) AS TrackCount FROM Tracks;`

Nesting Functions

- **Definition**: Combine functions for complex operations.
- **Example**:
- `SELECT UPPER(SUBSTR(FirstName, 1, 3)) AS ShortUpperName FROM Customers;`

Code Challenge: Customer Postal Code Transformation

- **Task**: Transform postal codes using string functions.
- **Example Solution**:
- `SELECT CustomerId, UPPER(SUBSTR(PostalCode, 1, 5)) AS ShortCode FROM Customers;`

8. Grouping

Grouping Your Query Results

- **Definition**: Use `GROUP BY` to aggregate data by unique values in a column.
- **Syntax**:
- `SELECT column, AGGREGATE_FUNCTION(column) FROM table_name GROUP BY column;`
- **Example**:
- `SELECT GenreId, COUNT(*) AS TrackCount FROM Tracks GROUP BY GenreId;`

Filtering with a Grouped Condition

- **Definition**: Use `HAVING` to filter grouped results.
- **Example**:
- `SELECT GenreId, COUNT(*) AS TrackCount FROM Tracks GROUP BY GenreId HAVING TrackCount > 100;`

Grouping with the WHERE Clause

- **Definition**: Apply `WHERE` before grouping to filter rows.
- **Example**:
- `SELECT GenreId, COUNT(*) FROM Tracks WHERE UnitPrice > 0.99 GROUP BY GenreId;`

Grouping with the HAVING Clause

- **Definition**: Apply `HAVING` after grouping to filter aggregated results.
- **Example**:
- ```
  SELECT GenreId, SUM(UnitPrice) AS TotalPrice FROM Tracks
  GROUP BY GenreId HAVING TotalPrice > 100;
  ```

Grouping with the WHERE and HAVING Clause

- **Definition**: Combine `WHERE` (pre-grouping filter) and `HAVING` (post-grouping filter).
- **Example**:
- ```
  SELECT GenreId, COUNT(*) AS TrackCount
  FROM Tracks
  WHERE UnitPrice > 0.99
  GROUP BY GenreId
  HAVING TrackCount > 50;
  ```

Grouping by Many Fields

- **Definition**: Group by multiple columns for more granular results.
- **Example**:
- ```
  SELECT GenreId, AlbumId, COUNT(*) AS TrackCount FROM Tracks
  GROUP BY GenreId, AlbumId;
  ```

Code Challenge: Calculate Average Spend per City

- **Task**: Calculate average invoice totals per city.
- **Example Solution**:
- ```
  SELECT c.City, AVG(i.Total) AS AvgSpend
  FROM Customers c
  JOIN Invoices i ON c.CustomerId = i.CustomerId
  GROUP BY c.City;
  ```

---

9. Nesting Queries

Subqueries and Aggregate Functions

- **Definition**: A subquery is a query nested inside another query, often used with aggregate functions.
- **Example**:

- ```
  SELECT TrackName
  FROM Tracks
  WHERE UnitPrice > (SELECT AVG(UnitPrice) FROM Tracks);
  ```

SELECT Clause Subquery

- **Definition**: Use a subquery in the `SELECT` clause to return a single value per row.
- **Example**:
- `SELECT TrackName, (SELECT AVG(UnitPrice) FROM Tracks) AS AvgPrice FROM Tracks;`

Aggregated Subqueries

- **Definition**: Subqueries that return aggregated values.
- **Example**:
- ```
  SELECT GenreId
  FROM Tracks
  GROUP BY GenreId
  HAVING COUNT(*) > (SELECT AVG(COUNT(*)) FROM Tracks GROUP BY
  GenreId);
  ```

Non-Aggregate Subqueries

- **Definition**: Subqueries that return non-aggregated data.
- **Example**:
- ```
  SELECT TrackName
  FROM Tracks
  WHERE AlbumId = (SELECT AlbumId FROM Albums WHERE Title =
  'Greatest Hits');
  ```

IN Clause Subquery

- **Definition**: Use a subquery with `IN` to filter rows.
- **Example**:
- ```
  SELECT TrackName
  FROM Tracks
  WHERE GenreId IN (SELECT GenreId FROM Genres WHERE Name LIKE
  'Rock%');
  ```

DISTINCT Clause Subquery

- **Definition**: Use `DISTINCT` in a subquery to avoid duplicate values.
- **Example**:
- ```
  SELECT CustomerId
  FROM Invoices
  WHERE InvoiceId IN (SELECT DISTINCT InvoiceId FROM Invoice_Items
  WHERE UnitPrice > 1.00);
  ```

Code Challenge: Uncovering Unpopular Tracks

- **Task**: Identify tracks not purchased (not in `Invoice_Items`).
- **Example Solution**:
- ```
  SELECT TrackName
  FROM Tracks
  WHERE TrackId NOT IN (SELECT TrackId FROM Invoice_Items);
  ```

---

10. Stored Queries

View Introduction

- **Definition**: A view is a saved SQL query that acts like a virtual table.
- **Syntax**:
- `CREATE VIEW view_name AS SELECT ...;`

Creating a View

- **Example**:
- ```
  CREATE VIEW CustomerInvoices AS
  SELECT c.FirstName, i.InvoiceId
  FROM Customers c
  JOIN Invoices i ON c.CustomerId = i.CustomerId;
  ```

Editing a View

- **Syntax**:
- `ALTER VIEW view_name AS SELECT ...;`
- **Note**: SQLite has limited support; often requires dropping and recreating.
  ```

Joining Views

- **Definition**: Treat views like tables in joins.
- **Example**:
- ```
  SELECT v.FirstName, t.TrackName
  FROM CustomerInvoices v
  JOIN Invoice_Items ii ON v.InvoiceId = ii.InvoiceId
  JOIN Tracks t ON ii.TrackId = t.TrackId;
  ```

Deleting Views

- **Syntax**:
- `DROP VIEW view_name;`

---

11. Adding, Modifying, and Deleting Data

Analysis and Administration

- **Definition**: SQL supports data manipulation (INSERT, UPDATE, DELETE) alongside querying.
- **Use Case**: Manage database content for updates or cleanup.

Inserting Data

- **Syntax**:
- `INSERT INTO table_name (column1, column2) VALUES (value1, value2);`
- **Example**:
- `INSERT INTO Customers (FirstName, LastName) VALUES ('Jane', 'Doe');`

Updating Data

- **Syntax**:
- `UPDATE table_name SET column1 = value1 WHERE condition;`
- **Example**:
- `UPDATE Customers SET Email = 'jane.doe@email.com' WHERE CustomerId = 1;`
  ```

Deleting Data

- **Syntax**:
- `DELETE FROM table_name WHERE condition;`
- **Example**:
- `DELETE FROM Customers WHERE CustomerId = 1;`